

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



PROYECTO FINAL
INFORME DE AVANCE DE PROYECTO: MICROPHOTO

ASIGNATURA
SISTEMAS DISTRIBUIDOS
GRUPO 2

DOCENTE
MOLINA BARRIGA, MARIBEL

INTEGRANTES
BEDREGAL PEREZ, DANIEL
JARA MAMANI, MARIEL ALISSON
MESTAS ZEGARRA, CHRISTIAN RAUL
NOA CAMINO, YENARO JOEL
SEQUEIROS CONDORI, LUIS GUSTAVO

AREQUIPA - PERÚ
2026

INFORME DE AVANCE DE PROYECTO: MICROPHOTO

1. Enlace a GitHub

<https://github.com/christianmz565/microphoto>

2. Introducción

Este informe de avance del proyecto Microphoto registra el estado actual del desarrollo, detalla los componentes implementados y señala las tareas pendientes.

3. Propósito

Este informe documenta el estado de avance técnico del proyecto al corte del 24 de junio de 2026, con el objetivo de:

- Comunicar el porcentaje de avance frente a lo planificado.
- Evidenciar los módulos entregables ya funcionales: backend distribuido (coordinador, worker, reaper), frontend web y configuración de infraestructura.
- Identificar las funcionalidades aún en desarrollo y proponer las acciones pendientes para el cierre del proyecto.

4. Alcance del informe

El informe cubre el periodo comprendido entre el inicio del proyecto y el 24 de junio de 2026. Se evalúan los siguientes módulos o áreas del sistema:

- Backend en Go (The Go Authors, 2009): coordinador HTTP, workers de procesamiento y reaper de tareas colgadas.
- Frontend web: aplicación Astro (Astro Contributors, 2026) con componentes React e integración via SSE.
- Infraestructura: configuración Docker Compose (Docker Authors, 2026) y manifiestos Kubernetes/Helm.
- Definición del esquema de mensajería con Protocol Buffers (Google, 2026) gestionado con la herramienta buf (Buf Technologies, Inc., 2026).

El avance se calcula a partir de los módulos funcionales del repositorio.

5. Descripción general

5.1. Contexto del proyecto

Microphoto es una plataforma web de procesamiento de imágenes que emplea una arquitectura de sistemas distribuidos para dividir, procesar en paralelo y reconstruir imágenes aplicando filtros digitales. El sistema resuelve el problema de los tiempos de procesamiento secuencial elevados en imágenes de gran tamaño, distribuyendo el trabajo entre múltiples nodos worker.

El proyecto persigue los siguientes objetivos técnicos:

- Implementar un pipeline de procesamiento distribuido que fragmente automáticamente las imágenes en subtareas, las distribuya entre workers disponibles y reconstruya la imagen final.
- Ofrecer retroalimentación en tiempo real al usuario sobre el estado del procesamiento mediante Server-Sent Events (SSE).

- Escalar horizontalmente el número de workers mediante réplicas en contenedores Docker o Kubernetes.

Los filtros disponibles en la versión actual son: escala de grises (GRAYSCALE), desenfoque gaussiano (BLUR), ajuste de brillo (BRIGHTNESS) y redimensionamiento (RESIZE).

5.2. Estado general del proyecto

El proyecto se encuentra en un avance aproximado del 50 %, con la arquitectura central completamente diseñada e implementada. Los tres servicios del backend están operativos y se comunican correctamente a través de Redis (Redis Ltd., 2026) y MinIO (MinIO Inc., 2026). El frontend cuenta con todas las vistas e integración SSE funcionales. La configuración de Docker Compose permite levantar el sistema completo de forma local. El despliegue en Kubernetes está parcialmente configurado, a falta de validación en un entorno real. Las pruebas automatizadas, la autenticación de usuarios y el monitoreo con Prometheus/ Grafana completo son funcionalidades pendientes de implementación.

6. Equipo y roles

6.1. Roles del equipo

Rol	Integrantes	Responsabilidades
Backend	Bedregal Perez, Daniel Mestas Zegarra, Christian Raul Noa Camino, Yenaro Joel	Diseño e implementación del servicio coordinador HTTP, orquestación de tareas, procesamiento de imágenes con bimg y libvips, lógica de slicing y reconstitución, y reaper de control de timeouts
Frontend	Jara Mamani, Mariel Alisson	Desarrollo de la aplicación con Astro y React, integración SSE y componentes UI con shadcn/ui
Infrastructure	Sequeiros Condori, Luis Gustavo	Configuración Docker Compose, manifiestos Kubernetes, Helm charts, Helmfile y gestión de secretos con SOPS

7. Backlog

7.1. Resumen de épicas

Código	Épica	Total HU	HU completadas	Avance
E-001	Pipeline de procesamiento distribuido de imágenes	8	8	100 %
E-002	Interfaz de usuario y experiencia de cliente	5	5	100 %
E-003	Observabilidad y monitoreo	3	1	33 %
E-004	Infraestructura y despliegue	4	2	50 %
E-005	Gestión de usuarios y seguridad	3	0	0 %

7.2. Historias de usuario principales

Las historias de usuario definen las necesidades y funcionalidades desde la perspectiva del usuario final:

Código	Historia de usuario	Descripción	Estado
HU-001	Subida y procesamiento de imágenes	Como usuario, deseo subir una imagen y configurar filtros para obtener una imagen editada según mis necesidades.	Completo
HU-002	Visualización de progreso en tiempo real	Como usuario, deseo ver el progreso del procesamiento en tiempo real para conocer cuánto falta para que termine la edición.	Completo
HU-003	Descarga de la imagen final	Como usuario, deseo descargar la imagen procesada en mi dispositivo para poder guardarla o utilizarla fuera de la plataforma.	Completo
HU-004	Resiliencia ante fallos del sistema	Como usuario, deseo que el procesamiento de mi imagen continúe de manera automática si ocurre una falla técnica, para evitar subir el archivo nuevamente.	Completo
HU-005	Historial local de tareas	Como usuario, deseo acceder a un historial de mis procesamientos recientes en la sesión actual para recuperar y descargar resultados anteriores.	Completo
HU-006	Página informativa de inicio	Como usuario, deseo ver una pantalla de inicio explicativa para comprender rápidamente el funcionamiento y las características de la plataforma.	Completo
HU-007	Registro e inicio de sesión	Como usuario, deseo crear una cuenta e iniciar sesión de forma segura para proteger mis datos y acceder a funciones personalizadas.	Pendiente
HU-008	Historial persistente en la cuenta	Como usuario, deseo que mi historial de imágenes se guarde de manera permanente en mi cuenta para consultarlo desde cualquier dispositivo.	Pendiente

8. Requerimientos

Los requerimientos del sistema han sido derivados a partir de las necesidades expresadas en las historias de usuario anteriores.

8.1. Requerimientos funcionales

Código	Requerimiento	Estado	Avance
RF-001	Subida de imágenes desde el navegador vía multipart/form-data	Completo	100 %
RF-002	Fragmentación automática de la imagen en sub tareas proporcionales al número de píxeles	Completo	100 %
RF-003	Procesamiento paralelo de fragmentos por múltiples workers	Completo	100 %
RF-004	Filtro escala de grises (GRAYSCALE)	Completo	100 %
RF-005	Filtro desenfoque gaussiano (BLUR) con radio configurable	Completo	100 %
RF-006	Filtro de brillo (BRIGHTNESS) con factor configurable	Completo	100 %
RF-007	Filtro de redimensionamiento (RESIZE) con ancho y alto configurables	Completo	100 %
RF-008	Reconstrucción automática de fragmentos procesados en imagen final	Completo	100 %
RF-009	Notificación de progreso en tiempo real al cliente via SSE	Completo	100 %
RF-010	Descarga del resultado procesado desde el frontend	Completo	100 %
RF-011	Reintento automático de tareas que exceden el tiempo máximo (reaper)	Completo	100 %
RF-012	Historial de tareas procesadas en el navegador	Completo	100 %
RF-013	Página de landing informativa con descripción del producto	Completo	100 %
RF-014	Autenticación y gestión de usuarios	Pendiente	0 %
RF-015	Almacenamiento persistente del historial de tareas en base de datos	Pendiente	0 %

8.2. Requerimientos no funcionales

Código	Atributo	Estado	Avance
RNF-001	Escalabilidad horizontal: los workers se pueden replicar sin modificar el código	Completo	100 %
RNF-002	Tolerancia a fallos: el reaper detecta workers caídos y reagenda las tareas	Completo	100 %

Código	Atributo	Estado	Avance
RNF-003	Contenerización: Dockerfile multi-stage para los tres servicios	Completo	100 %
RNF-004	Observabilidad: métricas OpenTelemetry (OpenTelemetry Authors, 2026) exportadas a Prometheus	En curso	60 %
RNF-005	Gestión de objetos: imágenes almacenadas en MinIO con política de ciclo de vida	Completo	100 %
RNF-006	Serialización eficiente: mensajería de tareas via Protocol Buffers sobre Redis	Completo	100 %
RNF-007	Despliegue en Kubernetes: manifiestos y Helm charts definidos	En curso	50 %
RNF-008	Seguridad de secretos: gestión con SOPS en el directorio kube/	En curso	40 %
RNF-009	Pruebas automatizadas: cobertura de pruebas unitarias e integración	Pendiente	0 %
RNF-010	Tiempo de respuesta: retroalimentación al usuario en menos de 2 s tras el inicio	Completo	100 %

9. Plan de implementación

9.1. Fases

Fase	Nombre	Duración estimada	Estado	Entregable visible
1	Diseño de arquitectura y esquema de mensajería	1 semana	Completa	Diagrama de arquitectura, definición Proto (jobs.proto)
2	Backend: Pipeline central (coordinador + worker + reaper)	3 semanas	Completa	Servicios ejecutables, Docker Compose funcional
3	Frontend: SPA e integración SSE	2 semanas	Completa	Aplicación web accesible en navegador
4	Infraestructura Kubernetes y observabilidad	2 semanas	En curso	Helm charts parcialmente configurados, métricas Prometheus
5	Pruebas, seguridad y despliegue	2 semanas	Pendiente	Suite de pruebas, autenticación, despliegue en clúster real

9.2. Cronograma

Fase	Inicio previsto	Fin previsto	Inicio real	Fin real	Avance
1	01/05/2026	07/05/2026	01/05/2026	07/05/2026	100 %
2	08/05/2026	28/05/2026	08/05/2026	28/05/2026	100 %
3	29/05/2026	11/06/2026	29/05/2026	12/06/2026	100 %
4	12/06/2026	25/06/2026	12/06/2026	(en curso)	50 %
5	26/06/2026	10/07/2026	—	—	0 %

10. Arquitectura propuesta

El sistema sigue un patrón productor-cola-consumidor con almacenamiento de objetos compartido. A continuación, se detalla la arquitectura global de la solución y, posteriormente, la arquitectura individual de sus componentes e infraestructura de despliegue.

10.1. Arquitectura Global del Sistema

La arquitectura global de Microphoto orquesta la interacción entre el navegador web, el proxy de red Traefik, el coordinador, la cola de Redis, los workers de procesamiento y el almacenamiento en MinIO.

10.1.1. Estructura y Componentes Centrales

A continuación, se detalla la topología de red y los flujos de datos principales entre los diferentes componentes del sistema:

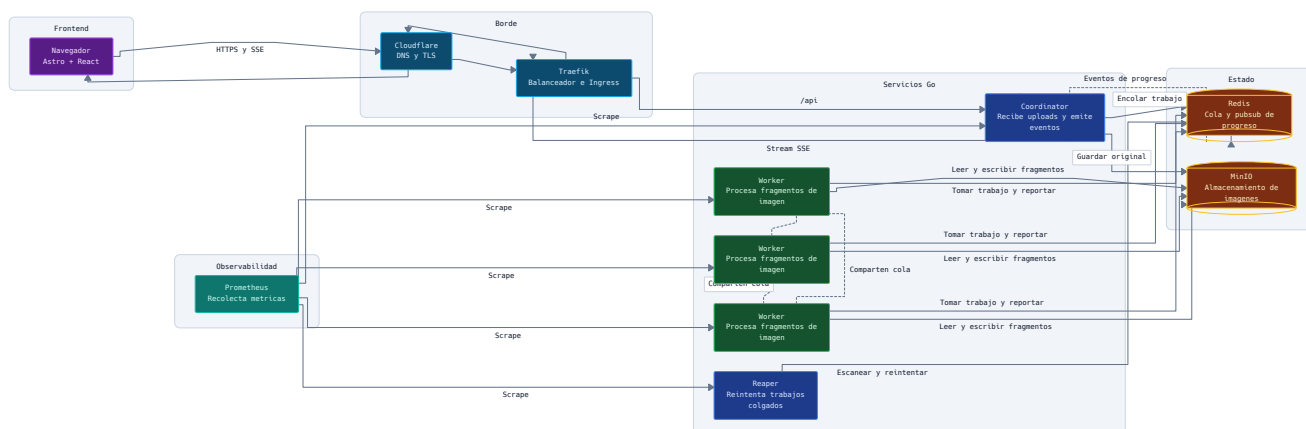


Figura 1. Diagrama de Arquitectura Global de Microphoto

10.1.2. Ciclo de Vida y Secuencia

El ciclo de vida de una tarea de procesamiento, desde la subida de la imagen y creación del trabajo de segmentación hasta la reconstrucción y entrega final, se describe cronológicamente en el diagrama de secuencia de la Figura 2.

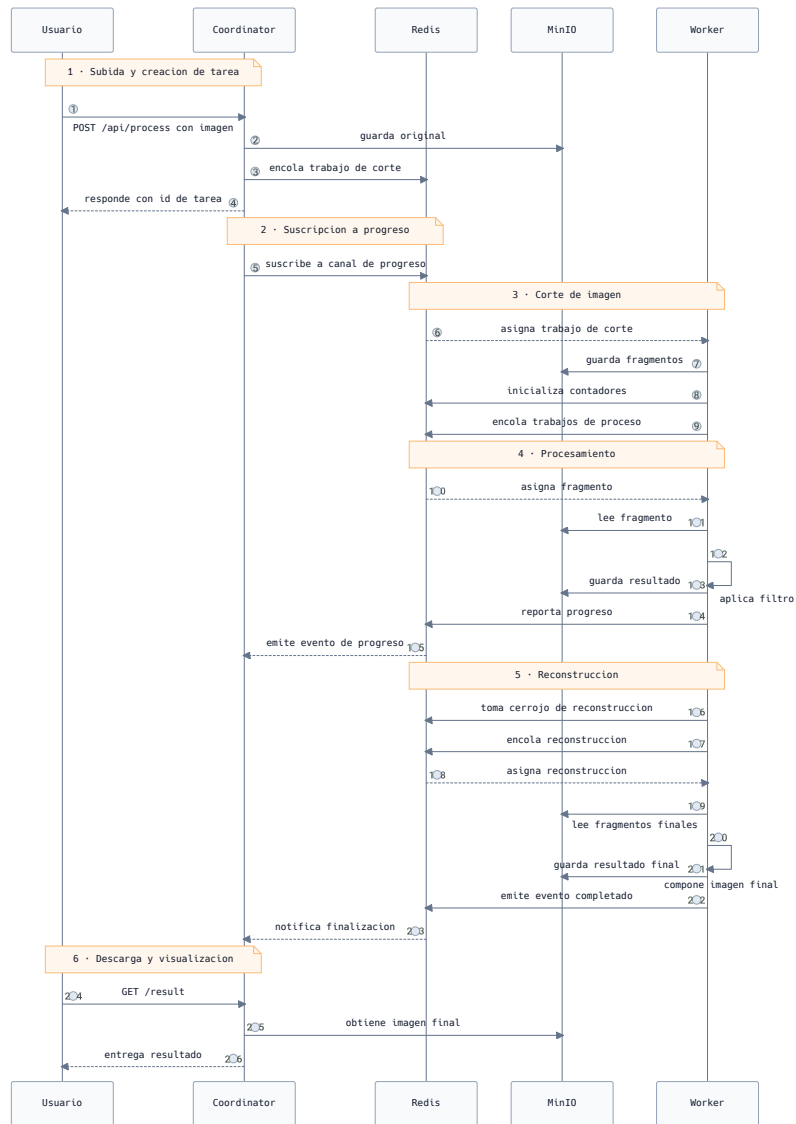


Figura 2. Diagrama de Secuencia Global del Ciclo de Vida de Procesamiento

10.1.3. Decisiones arquitectónicas clave

- Cola fiable con BLMOVE: el worker mueve atómicamente un job de {"global"}:queue a {"global"}:in_progress:{workerID}, de modo que si el worker falla, el reaper puede detectar el job huérfano y reagendarlo. Esto implementa el patrón de consumidores competidores (Hohpe & Woolf, 2003).
- Pub/Sub + historial de eventos: el progreso se publica en un canal Redis y también se persiste en una lista ({"global"}:events:{taskID}), lo que permite a los clientes que se conectan tarde recuperar el historial completo de eventos.
- Padding en fragmentos para blur: al dividir la imagen para aplicar el filtro de desenfoque, cada fragmento incluye filas adicionales de padding, las cuales se recortan después del procesamiento para evitar artefactos en los bordes, aplicando técnicas estándar de solapamiento (Rauber & Rünger, 2013).
- SetNX para reconstrucción única: un único worker dispara el job de reconstrucción usando SETNX en Redis, evitando que múltiples workers inicien la reconstitución simultáneamente cuando el contador llega a cero.

10.2. Arquitectura de Componentes e Infraestructura

10.2.1. Arquitectura del Frontend

La arquitectura del cliente web, implementada con Astro y React, se visualiza en la Figura 3. Se compone de componentes React interactivos montados sobre páginas estáticas de Astro, integrando el hook de eventos SSE y el historial local.

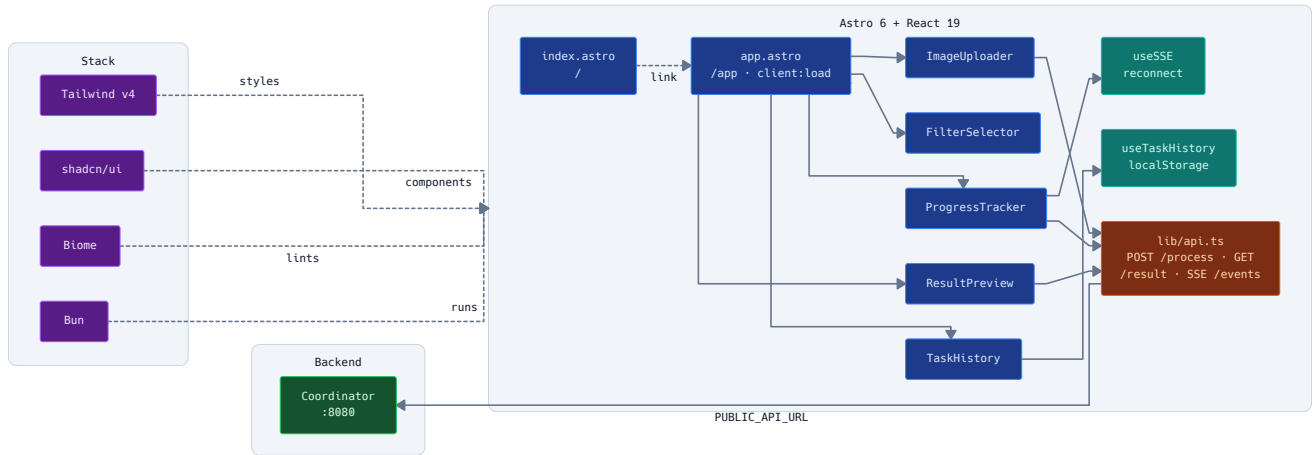


Figura 3. Diagrama de Componentes del Frontend de Microphoto

10.2.2. Despliegue e Infraestructura Kubernetes

Para entornos de producción y escalabilidad horizontal, se ha diseñado el esquema de infraestructura Kubernetes representado en la Figura 4, incorporando balanceo de carga, TLS y monitoreo mediante Prometheus y Node Exporter.

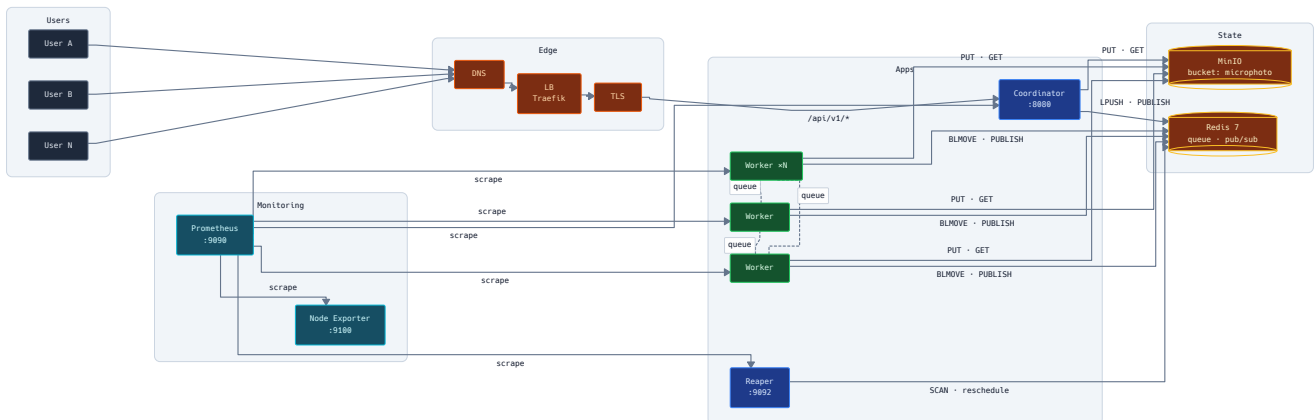


Figura 4. Diagrama de Despliegue en Kubernetes e Infraestructura

11. Avance actual

11.1. Trabajo completado

11.1.1. Backend

El backend está compuesto por tres servicios principales y una base de clientes compartida:

- **Coordinador:** gestiona las solicitudes de procesamiento de imágenes entrantes mediante un servidor HTTP, sube las imágenes originales a MinIO, divide la carga en subtareas empujándolas a la cola Redis y ofrece retroalimentación en tiempo real a través de Server-Sent Events (SSE) (Appelqvist, 2017).
- **Worker:** ejecuta el procesamiento paralelo consumiendo tareas de Redis. Realiza la segmentación de las imágenes en subtareas de un millón de píxeles, aplica filtros de escala de grises, brillo, desenfoque gaussiano con padding y redimensionamiento proporcional mediante bimg y libvips (Aparicio, 2020; Cupitt et al., 2025), y realiza la reconstrucción de la imagen resultante.
- **Reaper:** monitorea las tareas en proceso en Redis para detectar tiempos de expiración superados. Implementa una lógica de reintento automático y marcas de fallo definitivo cuando se agotan los intentos de procesamiento.
- **Clientes y Utilidades:** implementa wrappers de conexión para Redis y MinIO, incluyendo políticas de ciclo de vida para la expiración de datos temporales, y el soporte de métricas y telemetría de tareas para su consumo desde Prometheus.

11.1.2. Frontend

El frontend se estructura como una interfaz de usuario interactiva y responsiva:

- **Aplicación Principal:** controla el ciclo de estados de carga, procesamiento y previsualización de resultados mediante la integración con la API del coordinador.
- **Panel de Control:** incluye componentes para la subida de archivos mediante arrastrar y soltar, la selección y configuración detallada de parámetros de filtros, el seguimiento en tiempo real de workers activos con visualización de logs y un listado persistente con el historial local de tareas recientes.

11.1.3. Infraestructura

El entorno de despliegue abarca configuraciones locales y distribuidas:

- **Entorno Local:** utiliza Docker Compose para orquestar los contenedores de Redis, MinIO, coordinador, reaper y workers locales con capacidad de escalabilidad horizontal.
- **Entorno Kubernetes:** define manifiestos de red, ingress, gestión de certificados e integración con Helm charts y Helmfile (Helmfile Contributors, 2026) para gestionar despliegues de forma declarativa con secretos cifrados mediante SOPS (SOPS Contributors, 2026).

12. Evidencias

Se ha verificado el correcto funcionamiento del sistema en sus diferentes componentes:

- Código fuente en el repositorio: modulo Go verificado, dependencias declaradas y linter configurado.
- Artefactos de construcción: Dockerfile multi-stage con tres targets validados.
- Entorno local funcional: configuracion de Docker Compose que permite levantar el cluster local escalando el numero de workers.
- Esquema de mensajería: codigo generado a partir de la definición de Protocol Buffers mediante buf.
- Frontend compilable: compilacion estatica con bun run build.
- Capturas de la interfaz de usuario: las evidencias del funcionamiento visual de la interfaz se detallan en las figuras a continuación.



Figura 5. *Página de inicio informativa (Landing page) de Microphoto*

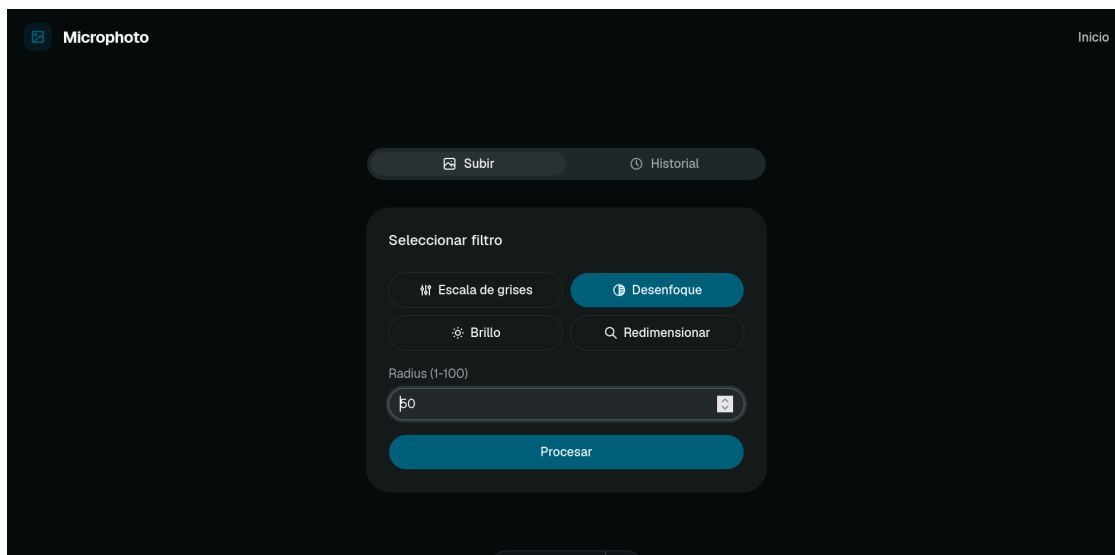


Figura 6. *Selección de filtros y parámetros configurables en la aplicación*

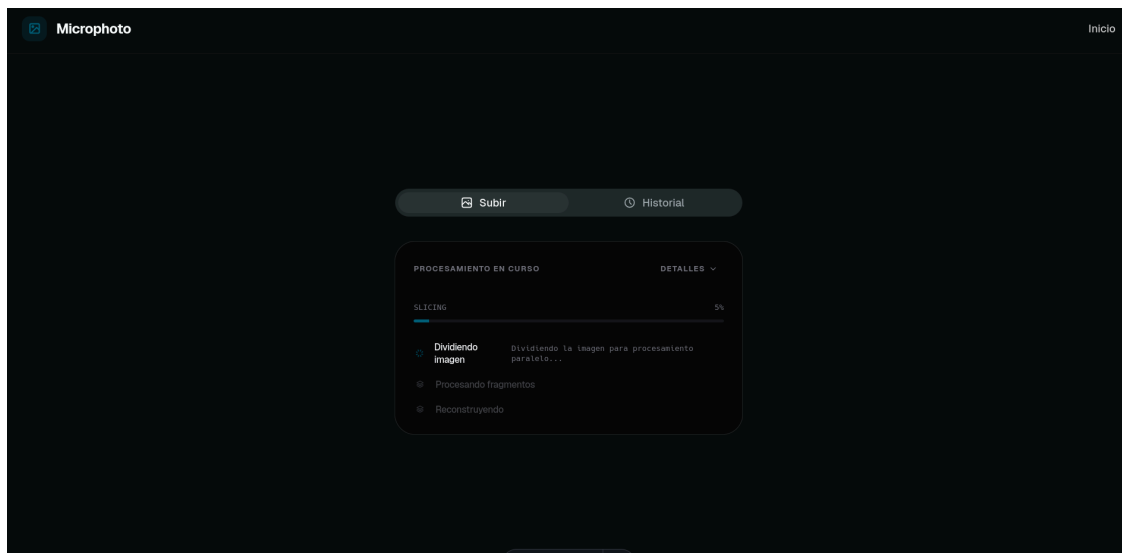


Figura 7. Progreso general de las fases de procesamiento distribuido

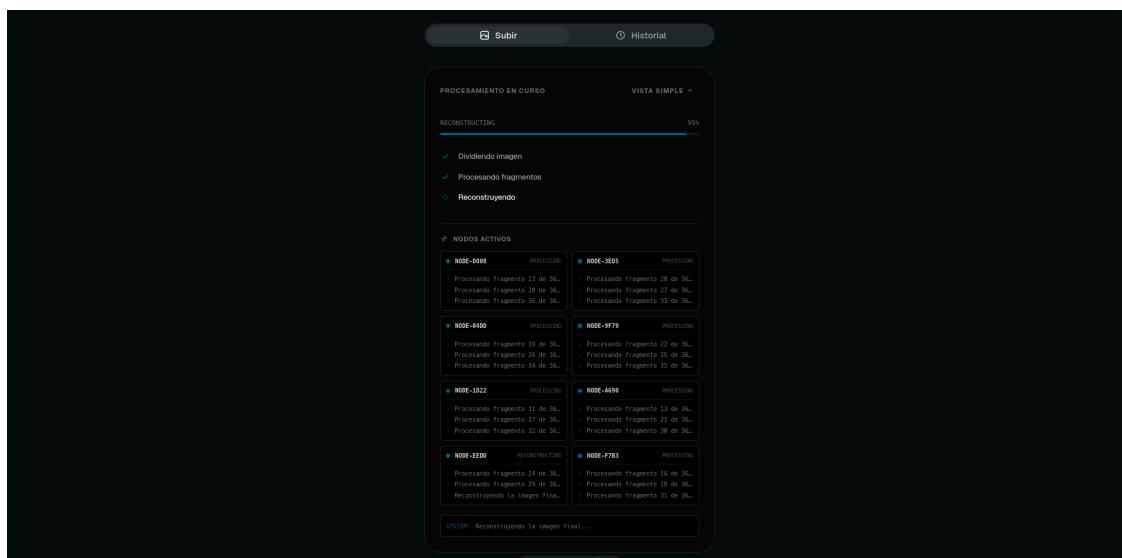


Figura 8. Detalle expandido con el log individual de cada nodo worker activo

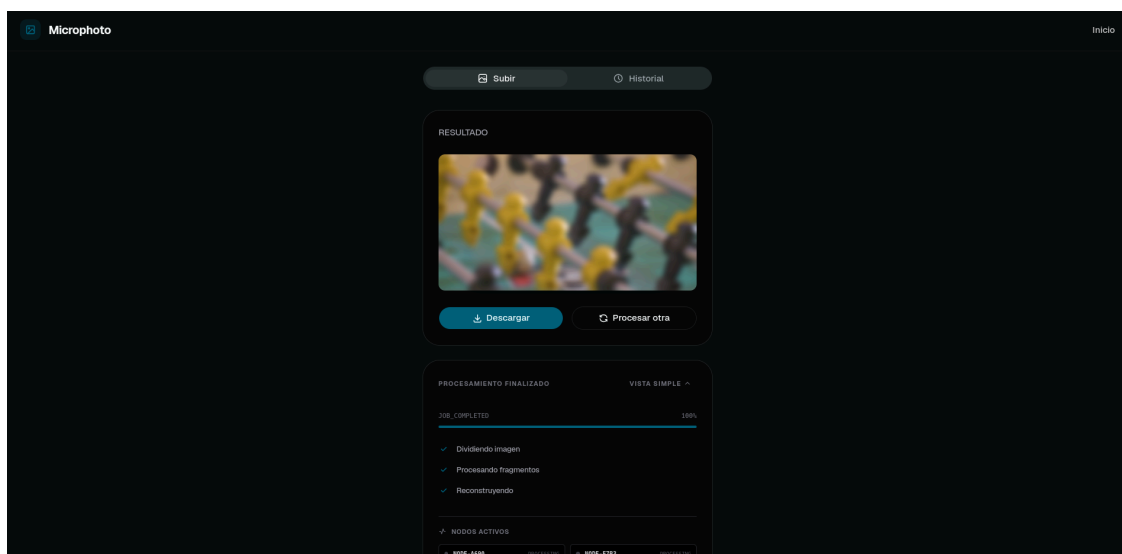


Figura 9. Previsualización de la imagen resultante procesada y opción de descarga

13. Referencias

- Aparicio, T. (2020,). *bimg: Go package for fast high-level image processing powered by libvips C library*. GitHub.
- Appelqvist, R. (2017). *Performance comparison of XHR polling, Long polling, Server sent events and Websockets*.
- Astro Contributors. (2026,). *Astro Documentation*.
- Buf Technologies, Inc. (2026,). *Buf Schema Registry and Tooling Documentation*.
- Cupitt, J., Martinez, K., Fuller, L., & Wolthuizen, K. A. (2025,). The libvips image processing library. *Proceedings of Electronic Imaging 2025*. <https://doi.org/10.2352/EI.2025.37.12.HPCI-178>
- Docker Authors. (2026,). *Docker Compose Specification*.
- The Go Authors. (2009,). *The Go Programming Language*.
- Google. (2026,). *Protocol Buffers*.
- Helmfile Contributors. (2026,). *Helmfile: Declarative Spec for Deploying Helm Charts*.
- Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional.
- MinIO Inc. (2026,). *MinIO Go Client SDK for Amazon S3 Compatible Cloud Storage*.
- OpenTelemetry Authors. (2026,). *OpenTelemetry Go Documentation*.
- Rauber, T., & Rünger, G. (2013). *Parallel Programming: for Multicore and Cluster Systems*. Springer Science & Business Media.
- Redis Ltd. (2026). *Redis Documentation*. <https://redis.io/docs/>
- SOPS Contributors. (2026,). *SOPS: Secrets Operations*.